

Forschungszentrum Jülich

---

# Technical Analysis and Research Roadmap for the FDS Slice-File Visualizer (fds\_visualizer)

---

A Code Audit, Scientific Visualization Assessment,  
and Improvement Plan for a Fire Dynamics Simulator Front-End

Prepared by: Samia Lachgar  
samia.lachgar@um6p.ma

July 5, 2026

## Abstract

This report is a full technical audit of the `fds_visualizer` repository, a small desktop application that plays back Fire Dynamics Simulator (FDS) slice-file (`.sf`) temperature fields as a looping, kiosk-style animation. The application was built as a public-outreach demonstrator (“Tag der Neugier”) rather than a research visualization tool, and this shows throughout its design: a single hard-coded scenario grid, a fixed  $1920 \times 1100$  full-screen layout, and no configuration, testing, or packaging infrastructure. The codebase is small (approximately 1,070 lines of Python across four files plus one Qt Designer form) but contains a genuinely reusable and reasonably well-engineered FDS binary-format parser (`src/fds/slice/slice.py`) buried underneath a much weaker GUI and data-loading layer. The report documents the architecture, traces data flow end-to-end, evaluates rendering and performance characteristics, and separates every claim into *Observed Fact*, *Interpretation*, and *Recommendation* categories. It concludes with a four-tier roadmap (Quick Wins, Medium, Major, Research-Level) intended as the starting point for an internship focused on visualization quality, rendering, responsiveness, data processing, and software architecture.

# Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                                      | <b>5</b>  |
| 1.1      | What this project actually is . . . . .                       | 5         |
| 1.2      | Key findings at a glance . . . . .                            | 5         |
| <b>2</b> | <b>Project Overview</b>                                       | <b>7</b>  |
| 2.1      | Repository layout . . . . .                                   | 7         |
| 2.2      | Provenance and intended use . . . . .                         | 7         |
| <b>3</b> | <b>Architecture Analysis</b>                                  | <b>9</b>  |
| 3.1      | Component diagram . . . . .                                   | 9         |
| 3.2      | Import / dependency graph . . . . .                           | 9         |
| 3.3      | Call graph of the hot path . . . . .                          | 10        |
| 3.4      | Sequence diagram: one playback frame . . . . .                | 10        |
| 3.5      | Class diagram . . . . .                                       | 10        |
| 3.6      | Data flow narrative . . . . .                                 | 10        |
| <b>4</b> | <b>Detailed Code Analysis</b>                                 | <b>14</b> |
| 4.1      | <code>src/fds/slice/slice.py</code> (687 lines) . . . . .     | 14        |
| 4.1.1    | Purpose and responsibilities . . . . .                        | 14        |
| 4.1.2    | Inputs / outputs . . . . .                                    | 14        |
| 4.1.3    | Main algorithms . . . . .                                     | 14        |
| 4.1.4    | Strengths . . . . .                                           | 15        |
| 4.1.5    | Weaknesses . . . . .                                          | 15        |
| 4.1.6    | Complexity, maintainability, scalability . . . . .            | 16        |
| 4.2      | <code>src/load_data.py</code> (39 lines) . . . . .            | 16        |
| 4.2.1    | Purpose . . . . .                                             | 16        |
| 4.2.2    | Main algorithm . . . . .                                      | 16        |
| 4.2.3    | Strengths . . . . .                                           | 16        |
| 4.2.4    | Weaknesses . . . . .                                          | 16        |
| 4.3      | <code>src/main.py</code> (277 lines) . . . . .                | 17        |
| 4.3.1    | Purpose . . . . .                                             | 17        |
| 4.3.2    | Class <code>Main(QMainWindow, Ui_MainWindow)</code> . . . . . | 17        |
| 4.3.3    | Class <code>UpdatePlot(QtCore.QThread)</code> . . . . .       | 17        |
| 4.3.4    | Strengths . . . . .                                           | 17        |
| 4.3.5    | Weaknesses . . . . .                                          | 18        |
| 4.3.6    | <code>src/main_window.ui</code> . . . . .                     | 18        |
| <b>5</b> | <b>Scientific Visualization Analysis</b>                      | <b>19</b> |
| 5.1      | How the scalar field is visualized . . . . .                  | 19        |
| 5.1.1    | Mathematical description of the rendering pipeline . . . . .  | 19        |
| 5.1.2    | Slice extraction and combination . . . . .                    | 19        |
| 5.1.3    | Animation and temporal navigation . . . . .                   | 20        |

|           |                                                                               |           |
|-----------|-------------------------------------------------------------------------------|-----------|
| 5.2       | GUI update mechanism and rendering backend . . . . .                          | 20        |
| <b>6</b>  | <b>Performance Analysis</b>                                                   | <b>21</b> |
| 6.1       | Computational and memory bottlenecks . . . . .                                | 21        |
| 6.2       | I/O bottlenecks . . . . .                                                     | 21        |
| 6.3       | GUI / concurrency bottlenecks and thread safety . . . . .                     | 22        |
| 6.4       | NumPy usage assessment . . . . .                                              | 22        |
| <b>7</b>  | <b>GUI Analysis</b>                                                           | <b>23</b> |
| 7.1       | Layout and navigation . . . . .                                               | 23        |
| 7.2       | Usability findings . . . . .                                                  | 23        |
| 7.3       | Missing features (from a scientific-visualization-tool perspective) . . . . . | 23        |
| <b>8</b>  | <b>Rendering / Graphics Quality Analysis</b>                                  | <b>24</b> |
| 8.1       | Current rendering quality . . . . .                                           | 24        |
| 8.2       | Opportunities and why they matter scientifically . . . . .                    | 24        |
| <b>9</b>  | <b>Simulation Visualization Improvements</b>                                  | <b>26</b> |
| <b>10</b> | <b>Software Engineering Review</b>                                            | <b>27</b> |
| 10.1      | Architecture and modularity . . . . .                                         | 27        |
| 10.2      | SOLID violations . . . . .                                                    | 27        |
| 10.3      | Testing, CI, configuration, error handling . . . . .                          | 27        |
| <b>11</b> | <b>Research Opportunities</b>                                                 | <b>29</b> |
| <b>12</b> | <b>Final Improvement Roadmap</b>                                              | <b>30</b> |
| 12.1      | Quick Wins . . . . .                                                          | 31        |
| 12.2      | Medium Improvements . . . . .                                                 | 32        |
| 12.3      | Major Improvements . . . . .                                                  | 32        |
| 12.4      | Research-Level Improvements . . . . .                                         | 33        |
| <b>13</b> | <b>Final Conclusions</b>                                                      | <b>35</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                      |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Component-level data flow. Everything left of <code>load_data.py</code> is reusable FDS-format infrastructure; everything from <code>load_data.py</code> rightward is exhibit-specific GUI glue. . . . .                                                                                                                                                             | 9  |
| 3.2 | Dependency graph. Orange nodes are project modules; grey nodes are third-party/stdlib dependencies. Several imports ( <code>matplotlib.pyplot</code> in two files, <code>scipy.interpolate</code> , <code>tarfile</code> , <code>re</code> , <code>matplotlib.animation</code> ) are dead imports with no call sites anywhere in the file that imports them. . . . . | 10 |
| 3.3 | Call graph of the critical path from process launch to the first steady-state animation frame. Steps <code>a1–a6</code> (bulk data loading) execute once, synchronously, and block the Qt event loop except for the single <code>app.processEvents()</code> call inside the loading loop. . . . .                                                                    | 11 |
| 3.4 | Sequence of one animation tick. The signal crosses from the worker thread to the GUI thread (Qt handles this marshalling automatically for queued connections), but the actual redraw ( <code>canvas.draw()</code> ) is a synchronous, full-figure matplotlib Agg-backend rasterization run on the GUI thread – it is not a partial blit update. . . . .             | 12 |
| 3.5 | Class diagram. <code>MeshCollection</code> and <code>SliceCollection</code> (simple list wrappers, omitted for space) complete the parser’s data model. . . . .                                                                                                                                                                                                      | 13 |

# List of Tables

|     |                                                                                                                                     |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | High-level verdict summary. . . . .                                                                                                 | 6  |
| 6.1 | Identified bottlenecks, their asymptotic behavior, and consequences. . . . .                                                        | 21 |
| 8.1 | Rendering and visual-quality improvement opportunities, ranked roughly by expected scientific value per engineering effort. . . . . | 25 |

# Chapter 1

## Executive Summary

### 1.1 What this project actually is

#### Observed Fact

The repository at `~/Desktop/fds_visualizer` contains exactly 6 Python files. Excluding the FDS simulation-generation script (`fds/generate_sim.py`, out of scope per the brief) and the empty `__init__.py`, the application logic totals:

- `src/main.py` — 277 lines — Qt GUI, event wiring, playback thread.
- `src/load_data.py` — 39 lines — eager bulk loader for all 24 simulation cases.
- `src/fds/slice/slice.py` — 687 lines — FDS `.smv/.sf` binary parser and slice-mesh mapper.
- `src/main_window.ui` — a Qt Designer XML form with absolute-pixel widget geometry.

There is no `requirements.txt`, `pyproject.toml`, `setup.py`, `README`, license file, test directory, CI configuration, or logging configuration file anywhere in the repository. The directory is also not a Git repository (`git log` run inside it fails – “not a git repository”).

#### Interpretation

This is a small, single-purpose exhibit application, not a general-purpose visualization tool. The header comment in `main.py` (“FDS SLCF Visualizer, FZ-Jülich, Max Boehler, Lukas Arnold, Version 1.0, 2019/06/28”) and the file `protocol/2019_05_21.tex`, which documents a  $2 \times 3 \times 3 \times 2 = 36$ -combination experiment design for a public-outreach event (“Tag der Neugier” – German for “Day of Curiosity”), confirm that the software was built to run unattended on a kiosk screen at a science-outreach event, looping a candle/door/vent scenario matrix. It was never intended as a maintained, extensible research tool, which explains the complete absence of packaging, testing, and configuration infrastructure. Everything in this report should be read with that origin in mind: the code is not badly engineered relative to its one-off, one-screen, one-event purpose – but it is far from what is needed to serve as the base of a research internship on visualization quality, rendering, and architecture, which is precisely why the internship exists.

### 1.2 Key findings at a glance

| Dimension                | Verdict                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core FDS parser          | Reasonably solid binary-format reverse-engineering of FDS <code>.sf/.smv</code> files; the most reusable asset in the repository. Weak on error handling and API design.                      |
| Data loading             | Eager, blocking, full-precision load of <i>all</i> 24 scenarios into a dense <code>float64</code> 4D array before the GUI opens. No caching, chunking, or lazy loading.                       |
| Rendering                | <code>matplotlib imshow</code> redrawn via <code>FigureCanvas.draw()</code> on every frame – a full-figure CPU redraw, not a GPU path, not a blit-based update.                               |
| Concurrency              | A single <code>QThread</code> driven by <code>time.sleep</code> pushes frames via a Qt signal; stopped with <code>QThread.terminate()</code> , which is unsafe and deprecated practice in Qt. |
| GUI                      | Fixed full-screen 1920×1100 layout with pixel-absolute widget placement; not resizable, not scalable to other displays, no keyboard/accessibility support.                                    |
| Scientific visualization | Single 2D scalar heat map, single quantity (temperature), single fixed slice, <code>gist_heat</code> colormap (not perceptually uniform), no vector fields, no 3D, no multi-view.             |
| Software engineering     | No tests, no CI, no logging framework use, no type hints beyond the FDS parser, no dependency manifest, no version control in the working copy.                                               |

Table 1.1: High-level verdict summary.



## Chapter 2

# Project Overview

### 2.1 Repository layout

#### Observed Fact

Directory tree (simulation binary payloads collapsed for readability; `fds/sim/` contains 24 scenario sub-directories, each holding FDS `.sf/.s3d/.smv/.out` output files and Slurm batch logs):

```
1  fds_visualizer/
2  |-- banner/                # print assets for the outreach event (out of scope)
3  |-- fds/
4  |   |-- generate_sim.py    # HPC batch-job generator (out of scope, see brief)
5  |   |-- template.fds      # FDS input deck template
6  |   |-- template_hvac.fds # FDS input deck template (HVAC variant)
7  |   |-- start_job.batch    # Slurm submission script
8  |   '-- sim/               # 24 pre-computed FDS simulation output folders
9  |       '-- c{0,1}_d{0,1}_vod{0,1,2}_voc{0,1}/
10 |           |-- *.smv      # Smokeview geometry/metadata file
11 |           |-- *.sf       # slice-file binary time series (parsed by slice.py)
12 |           |-- *.s3d(.sz) # 3D smoke/soot data (unused by this app)
13 |           |-- *_hrr.csv   # heat-release-rate time series (unused by this app)
14 |           |-- *_cpu.csv   # solver performance log (unused by this app)
15 |           '-- stdout/stderr/*.batch # HPC job artifacts
16 |-- image_video_sim/       # exhibit photos / rendered preview media
17 |-- protocol/              # LaTeX meeting note defining the 36-case design matrix
18 '-- src/
19 |   |-- main.py             # Qt application entry point + GUI controller + playback
20 |   |   thread
21 |   |-- load_data.py        # eager bulk loader over fds/sim/*
22 |   |-- main_window.ui      # Qt Designer form (absolute-geometry layout)
23 |   |-- logo/               # splash screen / window icon assets
24 |   '-- fds/
25 |       |-- __init__.py     # empty (not a real package init)
26 |       '-- slice/
27 |           '-- slice.py    # FDS .smv/.sf binary format parser + slice-mesh mapping
```

### 2.2 Provenance and intended use

#### Observed Fact

`main.py` lines 1–6 state: “FDS SLCF Vizualizer – FZ-Jülich – Max Boehler, Lukas Arnold – Version 1.0 – Date: 2019/06/28.” `protocol/2019_05_21.tex` defines a design matrix with axes *Anzahl der Kerzen* (candle count, 2 levels), *Türgeometrie* (door geometry, cited as 3 levels in the protocol text but only 2 levels – 0.050, 0.150 – are present

in `fds/generate_sim.py`), *Öffnung Decke am Eingang* (entrance ceiling opening: open/closed/ventilated, 3 levels) and *Öffnung Decke über Kerze* (candle ceiling opening: open/closed, 2 levels). The protocol document’s own table states “Anzahl Simulationen: 36”, while `generate_sim.py` and the actual `fds/sim/` directory both produce  $2 \times 2 \times 3 \times 2 = 24$  simulations.

### Interpretation

The mismatch between the planned 36-case matrix and the executed 24-case matrix indicates the door geometry axis was reduced from 3 to 2 levels between the 2019-05-21 planning meeting and final execution, without the protocol document being updated. This is a minor documentation inconsistency, but it is symptomatic of the project’s overall lack of any single source of truth: parameter counts are hard-coded independently in three places (`generate_sim.py`, `load_data.py`’s call `load_all_data(2,2,3,2,...)`, and implicitly in the folder-naming convention `c{c}_d{d}_vod{v}_voc{v}`), and nothing checks that they agree with each other or with what is actually on disk.

### Recommendation

Any refactor should introduce a single declarative scenario manifest (e.g. a JSON/YAML file listing the factor levels and the resulting folder names) that both the simulation generator and the visualizer read, eliminating the three-way hard-coded duplication described above.

# Chapter 3

## Architecture Analysis

### 3.1 Component diagram

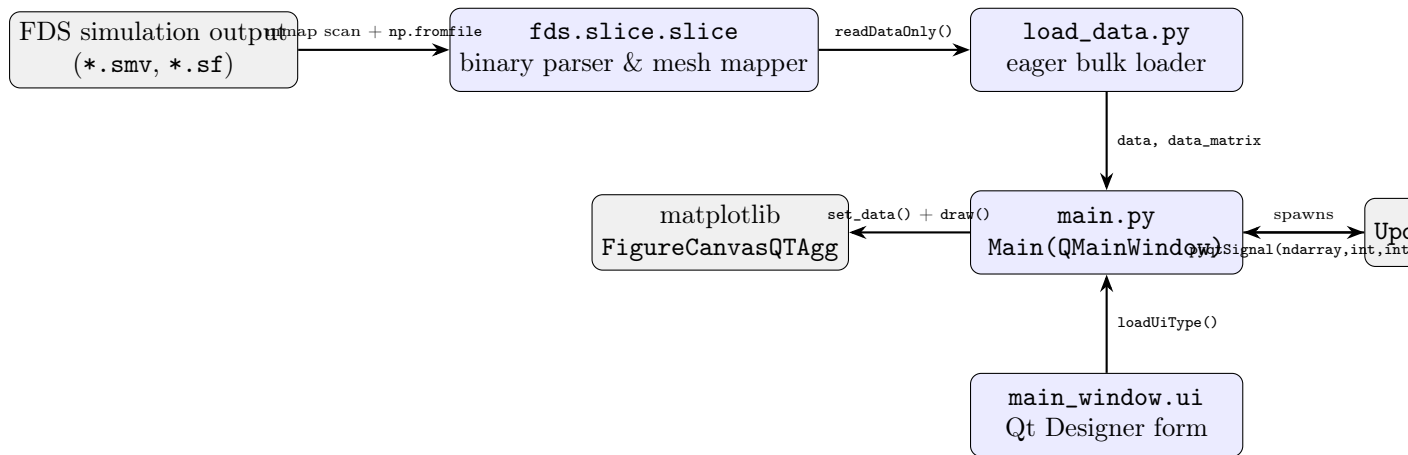


Figure 3.1: Component-level data flow. Everything left of `load_data.py` is reusable FDS-format infrastructure; everything from `load_data.py` rightward is exhibit-specific GUI glue.

### 3.2 Import / dependency graph

#### Observed Fact

Import edges extracted directly from the source files:

- `main.py` → `PyQt5` (`QtCore`, `QtWidgets`, `QtGui`, `uic`), `numpy`, `load_data`, `matplotlib.figure`, `matplotlib.backends.backend_qt5agg`, `time`, `sys`.
- `load_data.py` → `fds.slice.slice` (aliased `fds`), `numpy`, `matplotlib.pyplot` (imported but never used), `glob`, `os`.
- `fds/slice/slice.py` → `os`, `sys`, `re` (imported, never used), `tarfile` (imported, never used), `numpy`, `matplotlib.pyplot` (imported, never used), `scipy.interpolate` (imported, never used), `typing` (imported, only used for a single unenforced return-type hint), `matplotlib.animation` (imported, never used), `logging`, and locally `mmap`, `glob`, `os.path`.
- `fds/generate_sim.py` → `os` only.

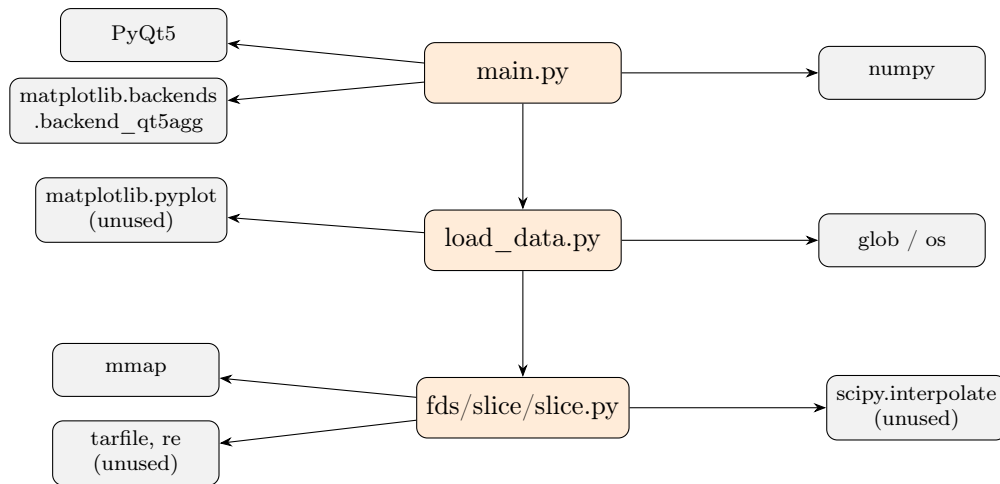


Figure 3.2: Dependency graph. Orange nodes are project modules; grey nodes are third-party/stdlib dependencies. Several imports (`matplotlib.pyplot` in two files, `scipy.interpolate`, `tarfile`, `re`, `matplotlib.animation`) are dead imports with no call sites anywhere in the file that imports them.

### 3.3 Call graph of the hot path

#### Observed Fact

Tracing from process start (`main.py` module-level code, lines 257–277) to the first frame on screen:

### 3.4 Sequence diagram: one playback frame

### 3.5 Class diagram

### 3.6 Data flow narrative

#### Observed Fact

**Where FDS data are loaded:** exclusively in `src/fds/slice/slice.py`, functions `readMeshes` (parses GRID/TRNX/TRNY/TRNZ records from the `.smv` text/binary file via a memory-mapped scan), `readSliceInfos` (parses SLC/ SLCC records for slice meta-data), and `Slice.readData/readAllTimes` (parses the actual `.sf` binary time series using `np.fromfile` with hand-built structured dtypes that encode Fortran unformatted-record framing, lines 20–42).

**Where preprocessing occurs:** `Slice.mapData` (reshapes the flat `data_raw` array into a (time, y, x) grid using the mesh geometry, lines 377–396) and `combineSlices` (stitches multiple per-mesh slices into a single global array when a quantity spans more than one FDS mesh, lines 431–478). `load_data.py:load_data` additionally does `np.flip(data, axis=1)` to correct for a vertical-axis convention mismatch between FDS’s mesh ordering and `imshow`’s default `origin='upper'` row order.

**Where rendering occurs:** exclusively in `src/main.py`, method `addmpl` (initial `Figure/imshow/colorbar` construction, lines 204–218) and `update_figure/update_figure_once` (per-frame `set_data` + `set_clim` +

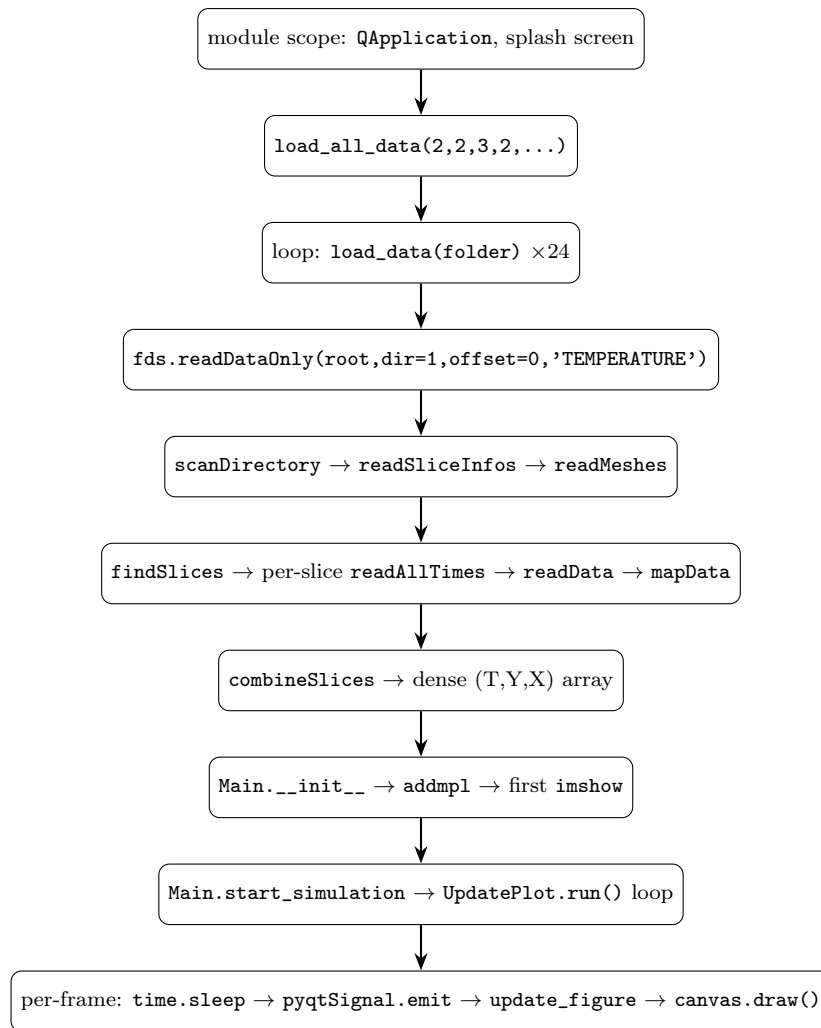


Figure 3.3: Call graph of the critical path from process launch to the first steady-state animation frame. Steps a1–a6 (bulk data loading) execute once, synchronously, and block the Qt event loop except for the single `app.processEvents()` call inside the loading loop.

`canvas.draw()`, lines 182–189 and 220–226).

**Where GUI logic resides:** entirely in `src/main.py` class `Main`, which mixes widget wiring, playback state machine, and rendering calls in one class with no separation of concerns, plus `src/main_window.ui` for static layout.

**Where visualization algorithms live:** there is exactly one visualization algorithm in the application – a 2D pseudocolor raster of a single scalar field via `matplotlib.imshow`. All "algorithmic" work (mesh reconstruction, slice combination, coordinate flipping) is data *preparation*, not visualization per se.

**Likely performance bottlenecks** (elaborated in Chapter 6): the eager, synchronous, full-precision load of all 24 scenarios at startup; the per-frame full-figure `canvas.draw()` call; and the `time.sleep`-based frame pacing combined with `QThread.terminate()` for stopping playback.

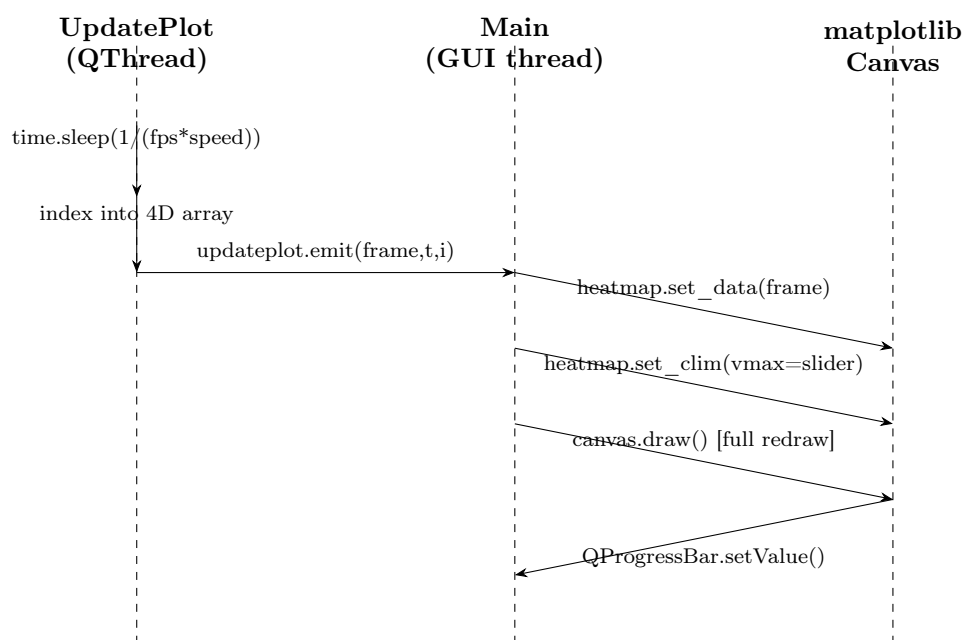


Figure 3.4: Sequence of one animation tick. The signal crosses from the worker thread to the GUI thread (Qt handles this marshalling automatically for queued connections), but the actual redraw (`canvas.draw()`) is a synchronous, full-figure matplotlib Agg-backend rasterization run on the GUI thread – it is not a partial `blit` update.

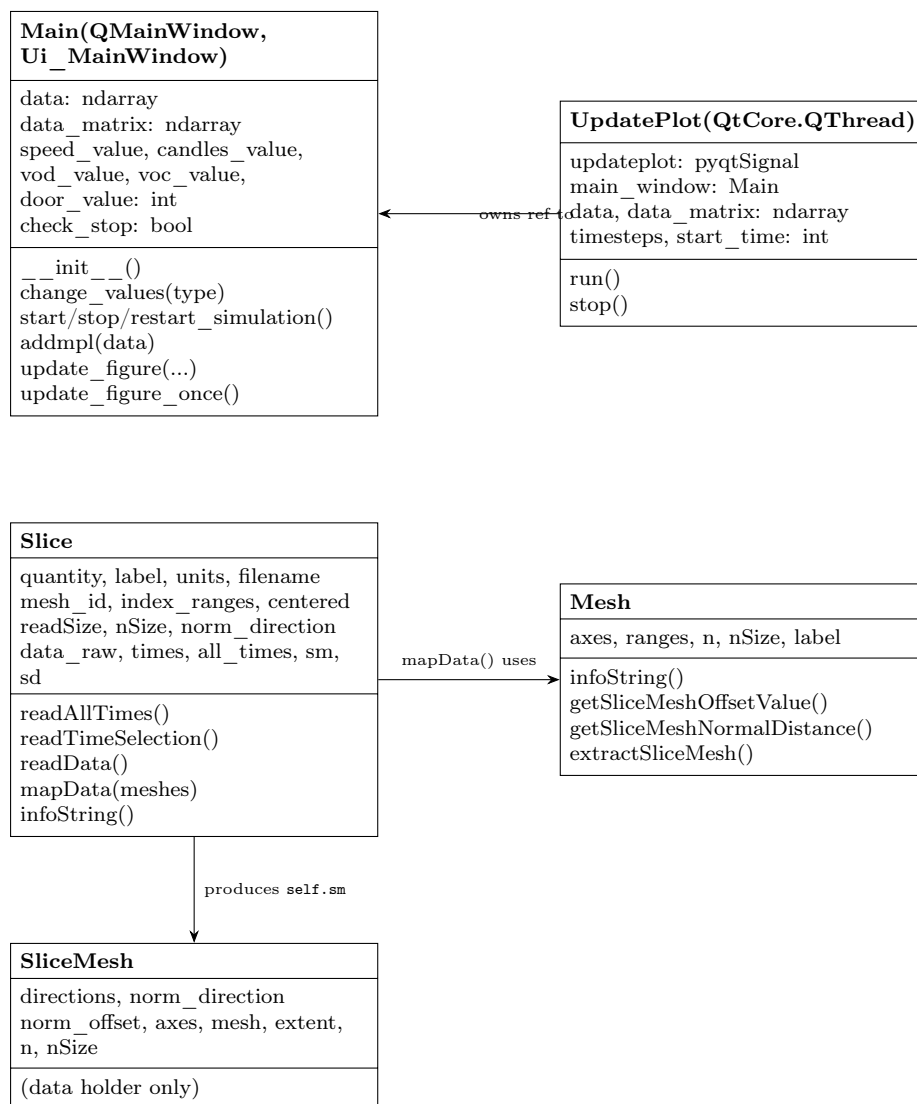


Figure 3.5: Class diagram. **MeshCollection** and **SliceCollection** (simple list wrappers, omitted for space) complete the parser’s data model.

## Chapter 4

# Detailed Code Analysis

### 4.1 src/fds/slice/slice.py (687 lines)

#### 4.1.1 Purpose and responsibilities

Implements a from-scratch reader for two FDS output formats: the ASCII/binary Smokeview geometry file (`.smv`) and the binary slice-file time series (`.sf`). It reconstructs mesh coordinate axes, slice metadata, per-timestep scalar fields, and can stitch slices from multiple FDS meshes into one combined array.

#### 4.1.2 Inputs / outputs

**Inputs:** a directory path containing one `.smv` file and one or more `.sf` files; a target quantity name (e.g. `'TEMPERATURE'`), a normal direction (`0/1/2` or `'x'/'y'/'z'`), and an offset index. **Outputs:** `readSlice` returns (`mesh`, `extent`, `data`, `mask`, `times`); `readDataOnly` (the function actually used by this application) returns only the dense `data` array of shape `(n_times, n_y, n_x)`.

#### 4.1.3 Main algorithms

- **Binary layout reconstruction** (`getSliceType`, lines 20–42): FDS writes Fortran unformatted sequential files, where each logical record is bracketed by a 4-byte integer record-length marker (controlled here by the `fds_fortran_backward` flag). The code builds `numpy` structured dtypes that read the leading marker, the payload, and the trailing marker in one `np.fromfile` call, which is an efficient and idiomatic way to parse Fortran binary output without a hand-rolled byte-offset parser.
- **Memory-mapped text scanning** (`readMeshes`, `readSliceInfos`, lines 135–194 and 480–520): uses Python’s `mmap` module with `bytes.find` to locate `GRID/TRNX/TRNY/TRNZ/SLC` tokens inside the `.smv` file without loading the whole file into a Python string, then `seek+readline` to pull fixed-format numeric fields that follow each token.
- **Random-access time-series reads** (`Slice.readData`, lines 338–373): computes a fixed `stride` per timestep record and seeks directly to each timestep’s payload, an  $O(n\_times)$  algorithm with  $O(1)$  seeks (good practice; avoids a full sequential parse).
- **Slice mosaicking** (`combineSlices`, lines 431–478): computes a bounding box over all matching slices from potentially different FDS meshes, allocates one dense output array, and copies each slice’s data into its sub-region using pointer arithmetic derived from cell size `dx1/dx2`.



#### 4.1.4 Strengths

- Uses `np.fromfile` with structured dtypes rather than manual byte unpacking – fast and idiomatic.
- `mmap`-based scanning of `.smv` avoids loading potentially large geometry files entirely into memory.
- Correctly separates "reading a slice" (`readData`) from "mapping onto a mesh" (`mapData`) from "combining multiple slices" (`combineSlices`) – the closest thing to good separation of concerns anywhere in the codebase.
- Domain knowledge is encoded precisely: comments about Fortran record framing and compiler-dependent binary layout (lines 14–18) show the author understood the fragility of the format and documented *why*, not just *what*.

#### 4.1.5 Weaknesses

- **No file handles are closed via context managers.** `readAllTimes`, `readTimeSelection`, and `readData` all call `open(...)` and rely on manual `infile.close()` (or, in `readAllTimes`, never close `slcf` at all – line 255–279 opens `slcf` and it is never closed). On an exception mid-parse, the file descriptor leaks.
- **No error handling anywhere.** Every `open()`, `np.fromfile`, and index lookup assumes well-formed input. `findSlices` returning an empty list is handled (`if len(slices) == 0: print(...); return None`), but a malformed `.smv` file, a truncated `.sf` file, or a mismatched quantity name will raise raw, unhandled exceptions (`IndexError`, `struct` errors) with no context about which file or slice failed.
- **Global mutable module state as configuration:** `fds_data_type_integer`, `fds_data_type_float`, `fds_fortran_backward` (lines 15–18) are module-level constants that silently assume a little-endian, 32-bit-int/float, backward-Fortran-record build of FDS. There is no version detection, no assertion, and no documented way to override them other than editing the source file.
- **Silent correctness assumption in `norm_direction`** (`Slice.__init__`, lines 240–248): if none of the three `if` conditions match (i.e. the slice is not axis-aligned on any of the three axes, which should not happen for FDS SLC records but is never validated), `self.norm_direction` is left undefined, and any later access raises an `AttributeError` far from the actual root cause.
- **Dead/unused imports:** `re`, `tarfile`, `matplotlib.pyplot`, `scipy.interpolate`, `matplotlib.animation`, `typing` (used once, superficially) are imported but never (or barely) used, inflating apparent dependencies and misleading readers about what the module actually needs.
- **`readSlice` and `readDataOnly` are near-duplicates:** lines 539–573 and 575–609 are copy-pasted verbatim except for the final return statement (one returns 5 values, the other only data). This is a direct SOLID/DRY violation (Section 10.2).
- **Commented-out block of ~80 lines at the end of the file** (lines 611–688) is dead code left in the module, indicating no cleanup pass was ever done before "shipping."
- **No type hints on the vast majority of functions/methods;** a handful of parameters (`norm_dir: str`, `filename`) have partial hints, but return types, class attributes, and most parameters are untyped.
- **`readData` always reads every stored timestep, and `readTimeSelection` exists but is never called by any code path used in this application** – meaning the "average over a time window" and "custom `dt` resampling" logic is currently unreachable dead functionality from the app's perspective, even though `load_data.py` always wants the same fixed quantity at full time resolution.

### 4.1.6 Complexity, maintainability, scalability

The parser is  $O(T \cdot N)$  per slice for reads ( $T$  timesteps,  $N$  cells per slice) and  $O(1)$  additional overhead per mesh/slice metadata scan (bounded by `.smv` file size). This is close to optimal for the problem. Maintainability is undermined by the duplicated `readSlice/readDataOnly` functions and lack of docstrings (only 3 inline explanatory comments exist in 687 lines). Scalability to larger domains is capped by the fact that `Slice.readData` allocates a dense `np.zeros((n_times, readSize)) float64` array up front (line 358) – for a simulation with many timesteps or a large slice this becomes a large contiguous allocation with no chunking or memory-mapped alternative.

## 4.2 `src/load_data.py` (39 lines)

### 4.2.1 Purpose

Bulk-loads temperature slice data for every simulation scenario in `fds/sim/` into one dense in-memory array and builds an index lookup table (`data_matrix`) from the four categorical scenario factors to a linear scenario index.

### 4.2.2 Main algorithm

`load_all_data(c, d, vod, voc, progressBar, app)` (lines 12–39):

1. Loads one scenario just to discover the array shape (line 13) – this means the very first scenario is parsed from disk *twice*: once to get `shape`, and again inside the main loop.
2. Counts sub-directories of `fds/sim/` to size a Qt progress bar (lines 16–19), iterating the directory listing a second time.
3. Iterates all scenario directories in sorted order, calling `load_data` per folder and writing into a pre-allocated array `data` of shape `(24, n_time, n_y, n_x)` at `float64` precision (line 14).
4. Builds `data_matrix`, a 4D integer array mapping `(candle, door, vod, voc) → linear index`, via a nested loop that simply counts upward (lines 30–36) – this assumes, without verifying, that `sorted(glob.iglob(...))` enumerates folders in exactly the same lexicographic order as the nested loop’s iteration order. Because the folder names are `c{i}_d{j}_vod{k}_voc{l}`, lexicographic sort of the string *does* coincide with the nested-loop counting order for single-digit indices – but this is an unstated invariant, not an enforced one.

### 4.2.3 Strengths

Minimal, does what it needs to for a fixed, known dataset; using a Qt `QProgressBar` plus `app.processEvents()` at least keeps the splash screen responsive during the otherwise blocking load (line 27).

### 4.2.4 Weaknesses

- **Hard-coded relative paths:** `'../fds/sim/c1_d0_vod0_voc0/'` (line 13) and `'../fds/sim/**'` (lines 16, 22) assume the process’s current working directory is `src/` at launch. There is no path resolution relative to the module’s own location (`os.path.dirname(__file__)`), so running the app from any other working directory silently fails or loads the wrong data.
- **Loads all 24 scenarios eagerly and unconditionally at startup**, at full `float64` precision, even though only one scenario is ever displayed at a time and the GUI can switch scenarios interactively via buttons. For the current dataset ( $24 \times 121 \times \sim 60 \times 40$ -ish grids) this is tolerable, but it is an  $O(\text{all scenarios})$  startup cost for an  $O(\text{one scenario})$

steady-state need, and it does not scale if the number of scenarios, resolution, or timesteps grows.

- **Silent broad except in the caller:** `main.py` wraps the call to `load_all_data` in a bare `try/except`: (lines 268–271) that catches *every* exception type and prints one generic "did you forget git-lfs" message, discarding the actual traceback. Any real bug in parsing, a missing folder, a permissions error, or an out-of-memory condition looks identical to the user.
- **Unused import:** `matplotlib.pyplot` (line 3) is never referenced.
- **Redundant double-scan of the directory:** the sub-directory count loop (lines 16–18) and the main load loop (line 22) both call `glob.iglob('../fds/sim/**')` independently instead of listing once and reusing the list.
- **Duplicate `progressBar.setValue(i)` calls** (lines 25 and 28) inside the same loop iteration – harmless but indicates the loop was edited without cleanup.
- **No caching to disk:** every application launch re-parses all 24 scenarios' full binary slice files from scratch; nothing is memoized (e.g. to a `.npy/HDF5/Zarr` cache keyed on file mtimes).

### 4.3 `src/main.py` (277 lines)

#### 4.3.1 Purpose

Application entry point: builds the `QApplication`, shows a splash screen while data loads, constructs the main window, and implements the GUI's event handling and playback control.

#### 4.3.2 Class `Main(QMainWindow, Ui_MainWindow)`

**Responsibilities (too many for one class):** Qt widget initialization; a five-factor scenario selector state machine (`speed`, `candles`, `vod`, `voc`, `door`) implemented via 16 near-identical `if/elif` branches in `change_values` (lines 77–160); animation lifecycle management (`start_simulation`, `stop_simulation`, `restart_simulation`); and direct matplotlib figure manipulation (`addmpl`, `update_figure`, `update_figure_once`). This is a textbook God Object: one class owns UI wiring, application state, business logic (which scenario maps to which array slab), *and* rendering.

#### 4.3.3 Class `UpdatePlot(QtCore.QThread)`

A worker thread whose `run()` method (lines 239–251) is an infinite `while True` loop that sleeps for `1/(timesteps*speed_value)` seconds, reads the currently selected scenario's next frame from the shared `data` array, and emits it via a Qt signal. Playback is stopped by calling `QThread.terminate()` (line 254), which forcibly kills the OS thread at an arbitrary point – Qt's own documentation calls this "a dangerous function" because it can leave the thread's resources (in this case, nothing critical, but this is not guaranteed for all future maintainers) in an undefined state, and it is explicitly discouraged in favor of a cooperative stop flag.

#### 4.3.4 Strengths

- Uses a background `QThread` plus a `pyqtSignal` to update the GUI, correctly avoiding direct cross-thread widget mutation (Qt widgets are not thread-safe, and this code does not touch them from the worker thread).
- The overall flow (splash → load → show window → start on demand) is a sensible shape for a kiosk application.

### 4.3.5 Weaknesses

- **change\_values** is a **16-branch if/elif chain keyed on a hard-coded string** (lines 77–160), setting `.setEnabled()` flags on button pairs/triples by hand. Adding one more factor level requires copy-pasting and hand-editing at least 3–4 more branches, and there is no exhaustiveness check – an unmatched `type` string silently falls through doing nothing.
- **A dangling reference bug:** `self.speed_1.setEnabled(False)` etc. at `__init__` time (lines 52–56) disable buttons *before* the corresponding `clicked` handlers are connected – functionally fine here, but the ordering is fragile and undocumented (a reader cannot tell whether this ordering matters without tracing every line).
- **Race condition risk between the worker thread and the main thread:** `UpdatePlot.run()` reads `self.main_window.speed_value`, `candles_value`, `door_value`, `vod_value`, `voc_value` every iteration (line 243) while the GUI thread can mutate those same attributes at any time via button clicks (`change_values`, called from the Qt event loop). Because these are plain Python `int` attribute reads/writes (atomic under the GIL) this does not currently corrupt memory, but it is an unsynchronized shared-mutable-state pattern that is fragile and would break under any richer state (e.g. a compound object) without warning.
- **stop\_simulation vs. start\_simulation leaks threads:** each call to `start_simulation/restart_simulation` constructs a brand-new `UpdatePlot` instance and calls `.start()` (lines 165–167, 193–198) without first waiting (`.wait()`) for a prior thread’s `terminate()` to actually finish. In principle this allows two `UpdatePlot` threads to be alive simultaneously, both holding references to `self.data`, both emitting signals to the same slot.
- **Hard-coded, absolute-pixel, full-screen UI:** `main_window.ui` specifies a fixed  $1920 \times 1100$  window and every widget has an absolute `<geometry> rect`; `main.py` calls `self.showFullScreen()` unconditionally (line 32) and re-enters full screen on any double-click (`mouseDoubleClickEvent`, lines 162–163). On any monitor other than exactly  $1920 \times 1080/1100$ , widgets will be cropped, misaligned, or leave dead space – there is no layout manager (`QVBoxLayout/QGridLayout`) used for the top-level widgets, only for the matplotlib canvas container (`mplv1`).
- **No way to exit full-screen except the on-screen “Exit” button**, and no keyboard shortcuts (e.g. Esc, F11) are wired – an accessibility and usability gap.
- `loadUiType('main_window.ui')` (line 26) and the icon/splash paths (`'logo/fds_vis_small.png'`, line 33; `'logo/fds_vis.png'`, line 260) are all relative paths resolved against the current working directory, not the module’s location – identical fragility to the one documented in `load_data.py`.
- **Global/module-level executable code** (lines 257–277) runs at import time with no `if __name__ == '__main__':` guard, meaning `main.py` cannot be imported (e.g. by a test suite or by another entry point) without immediately launching the full GUI application and blocking.
- **Duplicated code:** `start_simulation` and `restart_simulation` (lines 165–172, 193–202) are identical except for resetting `start_time = 1` first.

### 4.3.6 src/main\_window.ui

A Qt Designer XML form using `QWidgets` placed at absolute pixel coordinates rather than Qt layout managers (`QGridLayout`, `QFormLayout`), except for the single `QVBoxLayout` (`mplv1`) that hosts the matplotlib canvas. This is consistent with (and the direct cause of) the fixed-resolution assumption baked into `main.py`.

## Chapter 5

# Scientific Visualization Analysis

### 5.1 How the scalar field is visualized

#### Observed Fact

The application visualizes exactly one scalar field, FDS quantity 'TEMPERATURE' (`load_data.py`, line 8), extracted as a single axis-aligned 2D slice (`direction=1`, `offset=0`, i.e. a slice normal to the  $y$ -axis at mesh index 0). It is rendered with `matplotlib.axes.Axes.imshow(data, cmap='gist_heat')` (`main.py`, line 207) and a single shared colorbar (line 211). The colorbar upper limit is interactively adjustable via a `QSlider` (`temp`, range 30–300, mapped to `heatmap.set_clim(vmax=...)`), while the lower limit is left at matplotlib's automatic default from the first frame.

#### 5.1.1 Mathematical description of the rendering pipeline

Let  $T(x, y, t)$  be the discretized temperature field on the FDS slice mesh, of shape  $(n_y, n_x)$  at each of  $n_t$  discrete times. `imshow` maps each cell  $T_{i,j,t}$  to a color via a piecewise-linear normalization

$$n_{i,j,t} = \text{clip}\left(\frac{T_{i,j,t} - v_{\min}}{v_{\max} - v_{\min}}, 0, 1\right)$$

followed by a lookup into the `gist_heat` colormap function  $C : [0, 1] \rightarrow \mathbb{R}^3$  (RGB). Here  $v_{\min}$  is fixed at the first frame's data minimum (never updated), and  $v_{\max}$  is the live slider value (`self.temp.value()`), so *every subsequent frame is normalized against a stale, frame-0-derived  $v_{\min}$* . If later frames' minimum temperature drifts (physically plausible as the fire evolves and cooling occurs elsewhere in the domain), the color mapping's lower end silently becomes inaccurate.

#### 5.1.2 Slice extraction and combination

Mathematically, `Mesh.extractSliceMesh` projects the 3D mesh axes  $(x_1, x_2, x_3)$  onto the two axes orthogonal to `norm_dir`, producing a 2D mesh via `np.meshgrid`. When more than one FDS sub-mesh contributes to the same physical plane, `combineSlices` performs a mosaicking operation: it computes a global bounding box  $[\min x_1, \max x_1] \times [\min x_2, \max x_2]$  over all contributing slices' extents, assumes a uniform, shared cell size  $(dx_1, dx_2)$  taken from the *first* slice only (line 444–445 – this silently assumes all meshes share identical resolution, which is not verified), and copies each slice into its computed offset sub-region of one dense array. This is effectively nearest-neighbor tiling, not true interpolation: no resampling occurs across mesh boundaries, so any two meshes with different resolution would either misalign or (worse) silently overwrite adjacent data because the offset arithmetic (`int((s.sm.extent[0] - min_x1) / dx1)`) uses truncating integer division.

### 5.1.3 Animation and temporal navigation

#### Observed Fact

Animation is implemented as literal wall-clock-paced frame stepping: `UpdatePlot.run()` sleeps for  $1/(f \cdot s)$  seconds per frame, where  $f=4$  is the fixed `DT_SLCF`-derived frame rate (`self.timesteps = 4`, `main.py` line 43, hard-coded to match the FDS input deck's slice output interval of 0.25s) and  $s \in \{1, 2, 3\}$  is the user-selected speed multiplier. There is no scrubbing, no seek bar the user can drag, no pause-at-frame, and no reverse playback – `time_progress` (a `QProgressBar`, not a `QSlider`) is read-only display, not an interactive seek control, despite occupying the most prominent horizontal strip of the window.

#### Interpretation

Because frame pacing is `time.sleep`-based rather than driven by a Qt `QTimer` or a monotonic-clock-corrected scheduler, actual playback speed will drift under system load: each loop iteration's total period is sleep duration + (signal emission, GUI-thread redraw, and signal round-trip latency), none of which is accounted for. On a loaded system, or if `canvas.draw()` takes longer than one frame period (very plausible, see Chapter 6), frames will visibly stutter or the animation will fall behind real simulation time without any compensation.

## 5.2 GUI update mechanism and rendering backend

#### Observed Fact

Rendering uses `matplotlib`'s `Qt5Agg` backend (`FigureCanvasQTAgg`), which rasterizes the entire `matplotlib Figure` in software via the Anti-Grain Geometry (Agg) library on the CPU, then blits the resulting raster image into the Qt widget. There is no `pyqtgraph`, no `VisPy`, no raw `QOpenGLWidget`, and no GPU-accelerated rendering path anywhere in the codebase.

#### Interpretation

`canvas.draw()` (called every frame in `update_figure`, line 223) is `matplotlib`'s "redraw everything" API – it re-renders axes, colorbar, and image from scratch on every call, as opposed to `canvas.draw_idle()` combined with `blit` of only the changed `Artist` (the `AxesImage` returned by `imshow`). Because the colorbar and axes are static between frames (only the temperature array and the color-limit change), this is a clear case of a CPU-bound, non-vectorized-at-the-rendering-level bottleneck: every frame pays for re-rendering static content that never changes.

# Chapter 6

## Performance Analysis

### 6.1 Computational and memory bottlenecks

| Location                    | Issue                                                                                                                                             | Complexity                                | Consequence                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>load_all_data</code>  | Eager load of all 24 scenarios at <code>float64</code> before GUI opens; first scenario parsed twice (shape probe + main loop)                    | $O(S \cdot T \cdot N)$ , $S=24$ scenarios | Startup latency scales linearly with total scenario count and grid size; blocks event loop except for one <code>processEvents()</code> call |
| <code>Slice.readData</code> | Allocates <code>np.zeros((n_times, readSize))</code> at <code>float64</code> , one <code>np.fromfile</code> call per timestep (not one bulk read) | $O(T)$ syscalls, $O(T \cdot N)$ memory    | $T$ separate seek+read syscalls per slice instead of one strided/bulk read; doubles memory vs. <code>float32</code> source precision        |
| <code>combineSlices</code>  | Nearest-neighbor tiling with integer-truncated offsets, no interpolation                                                                          | $O(M \cdot N)$ , $M$ =meshes              | Silent misalignment risk if meshes have unequal cell size (unvalidated assumption)                                                          |
| <code>update_figure</code>  | Full <code>canvas.draw()</code> every frame instead of <code>blit</code>                                                                          | $O(\text{full figure raster})$ per frame  | GPU-bound per-frame cost dominated by re-rendering unchanged axes/colorbar chrome                                                           |
| <code>UpdatePlot.run</code> | <code>time.sleep</code> -paced infinite loop with no drift compensation                                                                           | –                                         | Playback speed is not guaranteed real-time-accurate under load                                                                              |

Table 6.1: Identified bottlenecks, their asymptotic behavior, and consequences.

### 6.2 I/O bottlenecks

- Every application launch re-reads and re-parses all 24 scenarios' `.smv` and `.sf` files from disk with no caching layer – for a "look at a fixed exhibit" use case this is repeated, avoidable I/O on every kiosk reboot.
- `Slice.readAllTimes` performs one `np.fromfile(count=1)` call per timestep in a Python-level `while True` loop (lines 264–279) to discover how many timesteps exist, rather than computing the count directly from file size and the known per-record stride – an easily-vectorizable operation left as a scalar Python loop.

### 6.3 GUI / concurrency bottlenecks and thread safety

- `QThread.terminate()` (main.py line 254) is documented by Qt as unsafe: it can be called at any point in the thread's execution, including mid-instruction, and provides no opportunity for cleanup. The correct pattern is a cooperative boolean flag checked each loop iteration plus `wait()` after requesting a stop.
- No mutex/lock protects the scenario-selection integers read by the worker thread and written by the GUI thread (Section 4.3, `UpdatePlot.run` vs. `Main.change_values`). Currently benign under CPython's GIL for simple int attributes, but it is an unsynchronized-shared-state anti-pattern.
- Starting a new `UpdatePlot` thread without confirming the previous one's `terminate()` has completed (no `.wait()`) creates a window where two threads can be alive and emitting to the same Qt slot concurrently.

### 6.4 NumPy usage assessment

The FDS parser's use of `np.fromfile` with structured dtypes is idiomatic and efficient. However, `Slice.readData`'s per-timestep loop (`for t in range(...): infile.seek(...); np.fromfile(...)`, lines 363–370) is a case where a single strided/bulk read (e.g. reading the whole file as one structured array with `count=n_times` using the known fixed stride, or using `np.memmap`) would replace  $T$  Python-level iterations and syscalls with one vectorized call. This is the clearest "non-vectorized operation that should be vectorized" in the codebase.



# Chapter 7

## GUI Analysis

### 7.1 Layout and navigation

#### Observed Fact

The interface is a single, non-resizable, full-screen window with no menus, tabs, or navigation of any kind (the `QMenuBar` in `main_window.ui` is present but empty – no `QActions` are defined). All 15 interactive widgets (5 scenario-factor button groups, 3 playback buttons, 1 slider, 1 progress bar) are visible simultaneously on one screen.

### 7.2 Usability findings

- **No way to know which scenario is currently selected at a glance** beyond which button is disabled (the "selected" state is encoded as `setEnabled(False)` on the chosen option, which is a strong overload of a control that normally means "unavailable," not "currently active" – a significant accessibility/legibility issue since a disabled (usually greyed-out) button is easy to misread as "this option cannot be chosen" rather than "this option is already chosen").
- **No numeric readout of simulation time**, only `time_progress` showing seconds via its `%v sec.` format string – functional but minimal.
- **No confirmation or visual feedback while data is loading** beyond the initial splash screen's progress bar; if loading takes long (e.g. on slower storage), the app gives no further indication that it has not frozen.
- **Restart requires stop-then-restart**, and `simulation_restart` is only enabled after `stop_simulation` is clicked (line 176) – there is no direct "restart from beginning while playing" action.
- **No keyboard interaction at all**: no shortcuts, no focus indicators, no way to operate the application without a mouse/touchscreen, which is a real limitation for an accessibility standard and even for a public kiosk (some venues require keyboard/switch-access alternatives).

### 7.3 Missing features (from a scientific-visualization-tool perspective)

No zoom/pan, no probe/read-out of the value under the cursor, no ability to change the visualized quantity or slice plane, no colorbar auto-rescale toggle, no screenshot/export function, no side-by-side comparison of two scenarios, no legend for what the button rows mean physically beyond static labels.

## Chapter 8

# Rendering / Graphics Quality Analysis

### 8.1 Current rendering quality

#### Observed Fact

Colormap: `gist_heat`, a non-perceptually-uniform, non-colorblind-safe matplotlib colormap (it is not one of matplotlib's "perceptually uniform sequential" maps such as `inferno`, `magma`, or `viridis`). Resolution: rendered at native slice-grid resolution with `imshow`'s default `interpolation='antialiased'` (matplotlib  $\geq 3.x$  default, i.e. mild automatic smoothing only, not a deliberate choice in this code). No anti-aliasing options are explicitly set. No lighting, shading, transparency, or depth cues are used because the visualization is strictly 2D.

### 8.2 Opportunities and why they matter scientifically

| Opportunity                                   | Current state                                            | Scientific justification                                                                                                                                                                  |
|-----------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Perceptually uniform colormap (inferno/magma) | <code>gist_heat</code> , uneven perceptual steps         | Equal data differences must map to equal perceived color differences, or viewers misjudge gradient steepness – critical for interpreting a temperature field’s spatial gradient correctly |
| Fixed, non-drifting normalization             | $v_{\min}$ frozen at frame 0, $v_{\max}$ user-set        | A static, physically-meaningful reference range (e.g. ambient to a fixed hazard threshold) avoids visually misrepresenting later frames                                                   |
| Bilinear/bicubic interpolation for display    | Native grid resolution, default matplotlib interpolation | Smoother apparent gradients better convey continuous physical fields without implying false grid-cell discreteness                                                                        |
| Streamlines / vector glyphs                   | Not present; only scalar temperature is loaded           | FDS also outputs velocity slices; visualizing flow direction alongside temperature reveals convective transport that a scalar field alone hides                                           |
| Clipping planes / multi-slice                 | Single fixed slice at <code>offset=0</code>              | A single 2D cut cannot show 3D fire plume structure; multiple parallel slices or an interactive cut plane reveal vertical stratification                                                  |
| Volume rendering                              | Not present                                              | FDS <code>.s3d</code> smoke/soot data exists on disk but is unused by this app; volumetric rendering would convey plume shape directly                                                    |
| Side-by-side multi-view                       | Single view of one scenario at a time                    | The dataset’s entire purpose is comparing 24 scenarios; a linked multi-panel view is the single highest-value visualization feature missing                                               |
| GPU-accelerated rendering                     | CPU Agg rasterization via matplotlib                     | Removes the per-frame full-redraw bottleneck and enables higher frame rates / larger grids without stutter                                                                                |

Table 8.1: Rendering and visual-quality improvement opportunities, ranked roughly by expected scientific value per engineering effort.

## Chapter 9

# Simulation Visualization Improvements

- **Lazy / on-demand loading:** load only the currently selected scenario (and prefetch adjacent ones) instead of all 24 up front. Rationale: the GUI can display only one scenario at a time (`data_matrix` selects a single index), so eager-loading the other 23 is pure wasted I/O and memory for every session; this directly shortens perceived startup time.
- **Disk-backed caching (e.g. `.npz`/`Zarr`/`HDF5`) keyed on source file mtimes:** avoids re-parsing binary `.sf` files on every launch when the underlying simulation output has not changed.
- **Chunked / memory-mapped reads (`np.memmap`) in `Slice.readData`:** replaces the current per-timestep Python loop and dense `float64` allocation with a lazily-paged view, reducing both peak memory and load latency, and enabling datasets larger than RAM.
- **Progressive/streaming load with immediate first-frame display:** show frame 0 as soon as it is parsed rather than waiting for the entire time series of the entire scenario matrix.
- **Blit-based animation instead of full `canvas.draw()`:** re-render only the `AxesImage` artist per frame; static chrome (axes, colorbar) is drawn once. This is a purely rendering-side fix requiring no changes to the data model.
- **Timer-based (`QTimer`) frame pacing instead of `time.sleep` in a `QThread`:** removes drift and the need for `terminate()`-based thread teardown, since a `QTimer` can simply be stopped.
- **Temporal interpolation between stored timesteps** is not currently needed (source data is evenly spaced at 0.25s and displayed at native resolution) but would become relevant if variable-rate playback or synchronization across scenarios of different simulated durations is added.

# Chapter 10

## Software Engineering Review

### 10.1 Architecture and modularity

#### Interpretation

The project has a reasonable *physical* split (parser vs. loader vs. GUI) but a poor *logical* one: `Main` in `main.py` concentrates widget wiring, a five-factor state machine, playback lifecycle, and direct matplotlib manipulation in a single 200-line class with no internal decomposition. There is no model/view separation of any kind: the "model" (which scenario is selected, at what time) and the "view" (matplotlib figure, Qt widgets) are updated by the same methods with no intermediate abstraction a test or a different front-end could target.

### 10.2 SOLID violations

- **Single Responsibility:** `Main` handles UI events, application state, and rendering. `Slice` handles both binary parsing (`readData`) and geometric remapping (`mapData`) – workable at this size, but already blurring the line.
- **Open/Closed:** adding a new scenario factor (e.g. a 5th binary variable) requires editing `change_values`'s `if/elif` chain, the `main_window.ui` layout, and `load_all_data`'s hard-coded shape/loop – nothing is closed to modification.
- **DRY (not strictly SOLID but closely related):** `readSlice/readDataOnly` in `slice.py` and `start_simulation/restart_simulation` in `main.py` are near-verbatim duplicates.

### 10.3 Testing, CI, configuration, error handling

#### Observed Fact

No test files, no test framework dependency, no CI configuration (no `.github/workflows`, no `.gitlab-ci.yml`), no `requirements.txt/pyproject.toml`, no `logging.getLogger(__name__)`-per-module pattern (only `slice.py` configures the root logger once, at `CRITICAL` level – effectively disabling all its own `logging.debug` calls by default), and exactly one broad, swallowing `except:` clause in the entire codebase (`main.py`, line 270).

**Recommendation**

Priority order for a research internship: (1) pin dependencies in a `pyproject.toml`; (2) extract the FDS parser into an installable, independently testable package with unit tests against known-good `.sf` fixtures; (3) introduce a thin `ViewModel/state` layer between `Main` and the plotting code so rendering logic becomes testable without a running Qt event loop; (4) replace the blanket `except:` with targeted exception handling that preserves the traceback in a log file.

## Chapter 11

# Research Opportunities

- **Multi-scenario linked comparison views:** since the dataset’s entire scientific purpose is comparing 24 parametric variations, a synchronized small-multiples or linked-cursor comparison view (shared time axis, shared color scale) is both a strong visualization-research topic (small-multiples design for parametric ensembles) and directly useful to the underlying fire-safety experiment.
- **Uncertainty / ensemble visualization:** treating the 24 runs as an ensemble and visualizing spread (e.g. min/max/variance envelopes over the candle/door/vent factors) at each spatial point is a recognized open problem in scientific visualization (ensemble uncertainty visualization).
- **GPU-accelerated large-slice rendering** (e.g. via VisPy/ModernGL/a QOpenGLWidget) as a testbed for comparing CPU-Agg vs. GPU throughput on real FDS grids – directly measurable, publishable performance-comparison research.
- **Automatic feature detection** (e.g. plume centerline, flame front, hot-layer interface via thresholding/connected components on the temperature field) turns passive visualization into an active analysis tool.
- **3D volumetric visualization of the unused .s3d smoke data** already present on disk but never read by this codebase – an entirely untouched data source ready for a volume-rendering research thread.

## Chapter 12

# Final Improvement Roadmap



## 12.1 Quick Wins

| Problem                                        | Current                                                                       | Solution                                                                                          | Effort  | Priority | Impact                                                  |
|------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|---------|----------|---------------------------------------------------------|
| Full-figure redraw per frame                   | <code>canvas.draw()</code> every frame                                        | Switch to <code>set_data</code> + <code>draw_idle</code> /blit on the <code>AxesImage</code> only | Low     | High     | Smoother, faster animation with zero data-model changes |
| Non-perceptual colormap                        | <code>gist_heat</code>                                                        | Switch to <code>inferno</code> / <code>magma</code>                                               | Low     | High     | Immediate, correct scientific color perception          |
| Unsafe thread stop                             | <code>QThread.terminate()</code>                                              | Cooperative stop flag + <code>wait()</code>                                                       | Low     | High     | Removes a latent crash/corruption risk                  |
| Hard-coded working-directory paths             | <code>'../fds/sim/...'</code> , <code>'logo/...'</code>                       | Resolve paths via <code>os.path.dirname(__file__)</code>                                          | Low     | High     | App runs regardless of launch cwd                       |
| Duplicated <code>readSlice/readDataOnly</code> | Two near-identical functions                                                  | Merge into one function with a <code>return_mesh=False</code> flag                                | Low     | Medium   | Removes a maintenance trap                              |
| Swallowing <code>except:</code>                | Generic “check git-lfs” message                                               | Catch specific exceptions, log full traceback                                                     | Low     | Medium   | Real bugs become diagnosable                            |
| Dead imports and dead code block               | Unused <code>re</code> , <code>tarfile</code> , etc.; 80-line commented block | Delete                                                                                            | Trivial | Low      | Cleaner, more trustworthy module surface                |

## 12.2 Medium Improvements

| Problem                                     | Current                                                | Solution                                                                                                  | Effort     | Priority | Impact                                          |
|---------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------|----------|-------------------------------------------------|
| Eager, full-precision load of all scenarios | 24 scenarios loaded at <code>float64</code> at startup | Lazy-load selected scenario + prefetch; <code>float32</code> storage                                      | Medium     | High     | Faster startup, lower memory                    |
| No model/view separation                    | <code>Main</code> owns state and rendering             | Introduce a <code>ScenarioState/ViewModel</code> class; <code>Main</code> only wires signals              | Medium     | High     | Enables unit testing and future UI reskins      |
| Resizable, responsive layout                | Fixed 1920x1100 absolute geometry                      | Rebuild <code>main_window.ui</code> with <code>QGridLayout/QVBoxLayout</code>                             | Medium     | Medium   | Works on any display; enables multi-panel views |
| No caching layer                            | Re-parses binary files every launch                    | Add an <code>.npy/HDF5</code> cache keyed on file mtimes                                                  | Medium     | Medium   | Near-instant subsequent launches                |
| No tests                                    | Zero test coverage                                     | Unit tests for <code>slice.py</code> parsing against fixture files; smoke test for <code>load_data</code> | Medium     | High     | Foundation for safely refactoring further       |
| No packaging                                | No <code>pyproject.toml/requirements.txt</code>        | Add dependency manifest + <code>src</code> layout package                                                 | Low–Medium | High     | Reproducible environment, installable parser    |

## 12.3 Major Improvements

| Problem                             | Current                                    | Solution                                                                                                  | Effort      | Priority | Impact                                                     |
|-------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------|----------|------------------------------------------------------------|
| Single-scenario, single-slice view  | One fixed 2D slice, one scenario at a time | Multi-panel linked comparison view (shared time/-color scale across scenarios)                            | High        | High     | Matches the dataset's actual scientific purpose            |
| CPU-bound rendering                 | matplotlib backend                         | Agg GPU-accelerated rendering via <code>pyqtgraph</code> / <code>VisPy</code> / <code>OpenGLWidget</code> | High        | Medium   | Higher frame rates, larger/more slices simultaneously      |
| No vector-field or 3D visualization | Only scalar temperature, 2D                | Add velocity slice glyphs/streamlines; add <code>.s3d</code> volumetric smoke rendering                   | High        | Medium   | Reveals flow structure and plume shape currently invisible |
| No interactive seek/scrub           | Read-only progress bar                     | Draggable <code>QSlider</code> timeline with random-access frame seeking                                  | Medium–High | Medium   | Turns passive playback into an analysis tool               |
| Monolithic GUI class                | Main God Object                            | Full MVC/MVVM refactor with a dedicated data-access layer reusing the parser as an installable package    | High        | High     | Long-term maintainability and extensibility                |

## 12.4 Research-Level Improvements

| Problem | Current | Solution | Effort | Priority | Impact |
|---------|---------|----------|--------|----------|--------|
|---------|---------|----------|--------|----------|--------|

|                                        |                                                 |                                                                                          |      |            |                                                   |
|----------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------|------|------------|---------------------------------------------------|
| No ensemble/uncertainty view           | 24 runs treated as independent, unrelated cases | Ensemble uncertainty visualization (spread/variance fields across the parametric matrix) | High | Medium     | Publishable scientific-visualization contribution |
| No automatic feature extraction        | Fully manual visual inspection                  | Automatic plume/flame-front/hot-layer detection on scalar fields                         | High | Low–Medium | Turns visualization into quantitative analysis    |
| No adaptive/multi-resolution rendering | Fixed native-resolution raster                  | Adaptive level-of-detail rendering for large grids/long time series                      | High | Low        | Scalability research relevant beyond this dataset |
| No AI-assisted analysis                | None                                            | ML-assisted anomaly/feature detection or scenario clustering across the 24-case ensemble | High | Low        | Novel, internship-appropriate research direction  |

## Chapter 13

# Final Conclusions

### Observed Fact

The codebase totals 1,070 lines of application Python across four files, has no tests, no CI, no dependency manifest, and is not currently under version control in the working copy. Its most valuable component, the FDS `.smv/.sf` binary parser, is competently written but undocumented, untested, and entangled with a single hard-coded, one-off GUI application built for a 2019 public science-outreach event.

### Interpretation

The project is best understood not as a mature visualization tool with a few bugs, but as a proof-of-concept exhibit demo whose data-access layer happens to be reusable. Every weakness identified above – the God Object GUI class, the eager full-dataset load, the full-figure redraw per frame, the absolute-pixel layout, the unsafe thread termination, the complete absence of tests – is consistent with software written once, for one screen, for one event, and never revisited. None of this is a moral failing of the original authors; it does mean that a research internship building on this code should treat the parser as a foundation to extract and harden, and treat almost everything else (GUI, data loading, rendering, playback) as a from-scratch redesign opportunity rather than an incremental-patch target.

### Recommendation

Recommended internship sequencing: (1) extract and test the FDS parser as an independent package; (2) introduce lazy loading, caching, and a proper model/view split; (3) fix the rendering pipeline (blit, perceptual colormap, GPU backend evaluation); (4) redesign the GUI with responsive layouts and an interactive timeline; (5) only then pursue the research-level visualization ideas (ensemble comparison, feature detection, volumetric rendering) in Chapter 9, which are the parts of this work most likely to yield a publishable contribution.